

Application Link: <http://nginx-web-app-alb-856331808.ap-south-1.elb.amazonaws.com/>

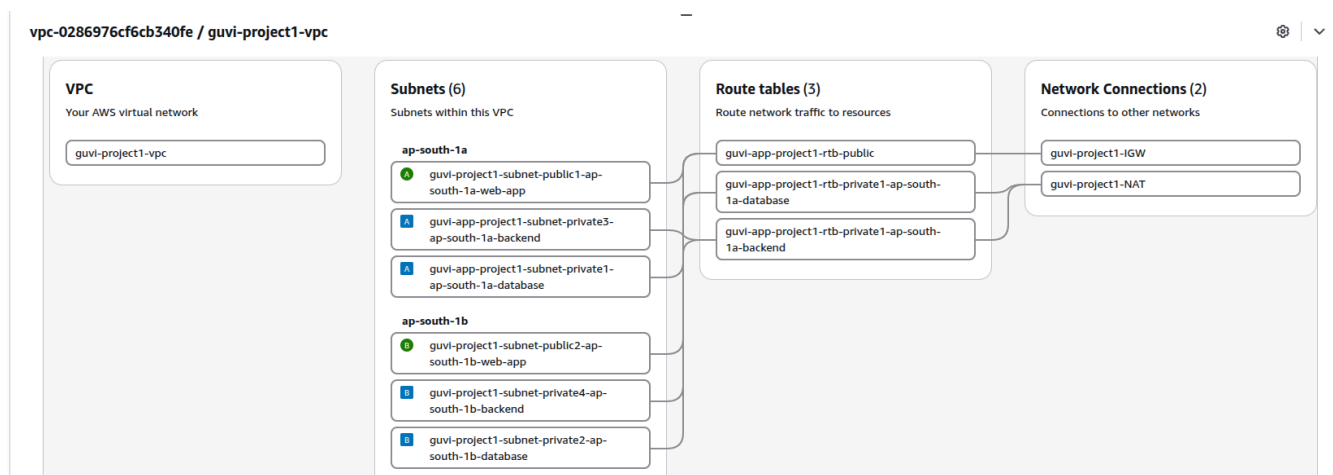
Task 1: Design and Configure the AWS Networking Infrastructure

This section details the successful deployment of the foundational AWS networking architecture required for the application. A **custom Virtual Private Cloud (VPC)** was configured with a well-planned CIDR block to securely host the infrastructure. To ensure high availability and fault tolerance, **6 subnets were provisioned and distributed evenly across 2 Availability Zones**. This architectural setup includes **2 Public Subnets** for the Web Tier, **2 Private Subnets** for the Application Tier, and **2 Private Subnets** for the Database Tier—with each subnet sized to support at least 256 IP addresses. For external connectivity, an **Internet Gateway (IGW)** was created and attached to the VPC to route public traffic.

Secure outbound internet access for the private tiers was established by deploying a **NAT Gateway (associated with an allocated Elastic IP)** within one of the Public Subnets.

Finally, traffic routing was effectively segregated by creating **three distinct Route Tables (Public, Private Application, and Private Database)**. These tables were accurately mapped to their respective subnets with the necessary routes configured for both internal communication and internet access.

The screenshots verifying the successful creation, configuration, and association of all the above resources are provided below.



NAT gateways (1) Info									
Find NAT gateways by attribute or tag									
☐	Name	NAT gateway ID	Connectivity...	State	State message	Availability ...	Route table ID	Primary public I...	Primary private I...
☐	guvi-project1-NAT	nat-0486c257406302b36	Public	Available	-	Zonal	-	13.200.183.206	10.0.3.124

Internet gateways (1) Info					
Find Internet gateways by attribute or tag					
☐	Name	Internet gateway ID	State	VPC ID	Owner
☐	guvi-project1-IGW	igw-0e7be9711c144436f	Attached	vpc-0286976cf6cb340fe guvi-project1-vpc	600402507299

Route tables (3) Info							
Find route tables by attribute or tag							
Last updated 3 minutes ago							
☐	Name	Route table ID	Explicit sub...	Edg...	Main	VPC	Owner ID
☐	guvi-app-project1-rtb-public	rtb-04ba4039eabd328f8	2 subnets	-	Yes	vpc-0286976cf6cb340fe guvi-project1-vpc	60040250...
☐	guvi-app-project1-rtb-private1-ap-south-1a-database	rtb-0bc16e396b8200b2a	2 subnets	-	No	vpc-0286976cf6cb340fe guvi-project1-vpc	60040250...
☐	guvi-app-project1-rtb-private1-ap-south-1a-backend	rtb-0a5d8c42826aafc7c	2 subnets	-	No	vpc-0286976cf6cb340fe guvi-project1-vpc	60040250...

Subnets (6) Info						
Find subnets by attribute or tag						
Last updated 2 minutes ago						
☐	Name	Subnet ID	State	VPC	Block Publi...	
☐	guvi-project1-subnet-private4-ap-south-1b-backend	subnet-032cab7b44490b9f	Available	vpc-0286976cf6cb340fe guvi-project1-vpc	☐ Off	
☐	guvi-project1-subnet-private2-ap-south-1b-database	subnet-07c98980a667cdb0b	Available	vpc-0286976cf6cb340fe guvi-project1-vpc	☐ Off	
☐	guvi-project1-subnet-public2-ap-south-1b-web-app	subnet-000b1eabc355dee4b	Available	vpc-0286976cf6cb340fe guvi-project1-vpc	☐ Off	
☐	guvi-project1-subnet-public1-ap-south-1a-web-app	subnet-0b7aa218ce03ac985	Available	vpc-0286976cf6cb340fe guvi-project1-vpc	☐ Off	
☐	guvi-app-project1-subnet-private3-ap-south-1a-backend	subnet-04b444f224fdb8e2d	Available	vpc-0286976cf6cb340fe guvi-project1-vpc	☐ Off	
☐	guvi-app-project1-subnet-private1-ap-south-1a-database	subnet-08df63ca9439a0124	Available	vpc-0286976cf6cb340fe guvi-project1-vpc	☐ Off	

Task 2: Deploy the Web Tier (Presentation Layer)

This section details the successful deployment and configuration of the application's Web Tier. To establish the presentation layer, **Amazon EC2 instances were provisioned and launched within the previously created Public Subnets**. Dedicated Security Groups were configured and attached to these instances to securely regulate inbound traffic. Following the infrastructure provisioning, secure connections were established to the instances to install the Nginx web server. **The provided HTML application files were then modified with some extra refinement by me and successfully copied and deployed**

to the server directories. Finally, the deployment was validated by confirming that the web application is fully accessible and rendering correctly via the public internet.

The screenshots verifying the EC2 instance creation, Security Group configurations, Nginx installation, and the live accessible web page are provided below.

sg-0b305cd088fc2feea - nginx-web-app-sg

Actions

Details

Security group name
nginx-web-app-sg

Security group ID
sg-0b305cd088fc2feea

Description
nginx-web-app-sg

VPC ID
vpc-0286976cf6cb340fe

Owner
600402507299

Inbound rules count
2 Permission entries

Outbound rules count
1 Permission entry

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (2)



Manage tags

Edit inbound rules

Search

<

1

>

☐	Name	Security group rule...	IP version	Type	Protocol	Port range	Source	Desc
☐	-	sgr-0e94d402f334a93c9	-	HTTP	TCP	80	sg-02e908a40ceb9c027 / web-app-ALB	-
☐	-	sgr-0415392cc33c5c3f	IPv4	SSH	TCP	22	122.162.149.166/32	-

Instance summary for i-0a7a97f0e242a17b7 (nginx-web-app) info



Connect

Instance state

Actions

Updated less than a minute ago

Instance ID

i-0a7a97f0e242a17b7

IPv6 address

-

Hostname type

IP name: ip-10-0-1-4.ap-south-1.compute.internal

Answer private resource DNS name

-

Auto-assigned IP address

3.6.94.194 [Public IP]

IAM role

-

IMDSv2

Required

Operator

-

Public IPv4 address

3.6.94.194 | open address

Instance state

Running

Private IP DNS name (IPv4 only)

ip-10-0-1-4.ap-south-1.compute.internal

Instance type

t3.micro

VPC ID

vpc-0286976cf6cb340fe (guvi-project1-vpc)

Subnet ID

subnet-0b7aa218ce03ac985 (guvi-project1-subnet-public1-ap-south-1a-web-app)

Instance ARN

arn:aws:ec2:ap-south-1:600402507299:instance/i-0a7a97f0e242a17b7

Private IPv4 addresses

10.0.1.4

Public DNS

ec2-3-6-94-194.ap-south-1.compute.amazonaws.com | open address

Elastic IP addresses

-

AWS Compute Optimizer finding

Opt-in to AWS Compute Optimizer for recommendations. | Learn more

Auto Scaling Group name

-

Managed

false

Not secure nginx-web-app-alb-856331808.ap-south-1.elb.amazonaws.com

DevOps Road... Osho Tantra M... Vigyan Bhairav... litu54/DevOps...

Submit Feedback

Your Name

Your Email

Your Feedback

Submit

Task 3: Deploy the Application Tier (Backend API)

This section outlines the deployment and configuration of the backend Application Tier. To host the backend logic, **Amazon EC2 instances were provisioned securely within the Private Application Subnet**. Secure remote access to these private instances was established **via a Bastion Host**. The runtime environment was configured by installing Python 3, Flask, and all required package dependencies as specified in the project repository. The primary application code (`app.py`) was then deployed to the instances. **Notably, the Python application was enhanced by implementing a custom GET API endpoint designed specifically to fetch feedback data**. Finally, the Flask application was **successfully initialized on port 5000**, and its operational status and internal connectivity were **verified**.

The screenshots verifying the private EC2 instance creation, secure connection setup, dependency installation, and the running Flask application (including the GET API response) are provided below.

```
ubuntu@ip-10-0-44-194:~$ curl -X POST http://localhost:5000/submit \
-H "Content-Type: application/json" \
-d '{
  "name": "Test",
  "email": "t@t.com",
  "feedback": "Hello RDS!"
}'
{"message": "Feedback submitted successfully!"}
ubuntu@ip-10-0-44-194:~$ _
```

Instance summary for i-0b6044a41de38fc0c (python-backend-api) info		
Updated less than a minute ago		
Instance ID i-0b6044a41de38fc0c	Public IPv4 address -	Private IPv4 addresses 10.0.44.194
IPv6 address -	Instance state Running	Public DNS -
Hostname type IP name: ip-10-0-44-194.ap-south-1.compute.internal	Private IP DNS name (IPv4 only) ip-10-0-44-194.ap-south-1.compute.internal	Elastic IP addresses -
Answer private resource DNS name -	Instance type t3.small	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto-assigned IP address -	VPC ID vpc-0286976cf6cb340fe (guvi-project1-vpc)	Auto Scaling Group name -
IAM role -	Subnet ID subnet-032cab7b44490b9f (guvi-project1-subnet-private4-ap-south-1-backend)	Managed false
IMDSv2 Required	Instance ARN arn:aws:ec2:ap-south-1:600402507299:instance/i-0b6044a41de38fc0c	
Operator -		

Task 4: Deploy and Configure the Database Tier

This section details the provisioning and configuration of the secure data persistence layer. A managed **Amazon RDS MySQL instance was created and strategically deployed within the isolated Private Database Subnets to ensure maximum security.** Database credentials and access parameters were configured according to the project requirements. Following the infrastructure setup, **a secure connection to the database was established directly from the Application Tier (EC2 terminal).** The provided SQL commands were then executed via the terminal to initialize the database schema and structure. Finally, seamless end-to-end backend communication was validated by **successfully verifying the connection between the Python Flask application and the MySQL database.**

The screenshots verifying the Amazon RDS instance creation, database subnet deployment, and successful Flask-to-Database connectivity are provided below.

```
ubuntu@ip-10-0-44-194:~$ mysql -h guvi-project1-mysql-db.c7q0e0ku8lf1.ap-south-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 310
Server version: 8.4.9 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE feedbackdb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SELECT * FROM feedback;
```

Databases (1) Group resources Modify Actions Create database

Filter by databases

DB identifier	Status	Role	Engine	Upgrade rollout ...	Region ...	Size	Recommendation
guvi-project1-mysql-db	Available	Instance	MySQL Community	SECOND	ap-south-1a	db.t4g.micro	

Aurora and RDS > **Databases** > guvi-project1-mysql-db

guvi-project1-mysql-db Modify Actions

Summary

DB identifier guvi-project1-mysql-db	Status Available	Role Instance	Engine MySQL Community	Recommendations
CPU 4.04%	Class db.t4g.micro	Current activity 0 Connections	Region & AZ ap-south-1a	

Connectivity & security | Monitoring | Logs & events | Configuration | Zero-ETL integrations | Maintenance & backups | Data migrations | Tags | Recommendations

Connect using [Info](#)

```
ubuntu@ip-10-0-44-194:~$ curl -X POST http://localhost:5000/submit \
-H "Content-Type: application/json" \
-d '{
  "name": "Test",
  "email": "t@t.com",
  "feedback": "Hello RDS!"
}'
{"message": "Feedback submitted successfully!"}
ubuntu@ip-10-0-44-194:~$ _
```

Task 5: Implement Network and Instance Security

This section outlines the implementation of robust network and instance-level security across the architecture. **Dedicated Security Groups were created and configured with strict inbound and outbound rules to enforce the principle of least privilege.** Custom security configurations were applied independently across all architectural layers, specifically for the Load Balancer, Web Tier, Application Tier, and Database Tier. To ensure secure internal communication and protect the infrastructure against unauthorized access, **network traffic was tightly restricted, ensuring that only the strictly required ports are permitted** to communicate between the respective tiers.

The screenshots verifying the precise inbound and outbound rule configurations for all the respective Security Groups are provided below.

sg-02e908a40ceb9c027 - web-app-ALB

Actions

Details

Security group name
web-app-ALB

Security group ID
sg-02e908a40ceb9c027

Description
web-app-ALB

VPC ID
[vpc-0286976cf6cb340fe](#)

Owner
600402507299

Inbound rules count
2 Permission entries

Outbound rules count
1 Permission entry

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (2)



Manage tags

Edit inbound rules

Search

< 1 > ⚙

☐	Name	Security group rule...	IP version	Type	Protocol	Port range	Source	Description
☐	-	sg-08f0f5f626fd247bb	IPv4	HTTPS	TCP	443	0.0.0.0/0	-
☐	-	sg-084786ff81cef974c	IPv4	HTTP	TCP	80	0.0.0.0/0	-

sg-004ddcb918f8a6f71 - python-app-ALB-SG

Actions

Details

Security group name
python-app-ALB-SG

Security group ID
sg-004ddcb918f8a6f71

Description
python-app-ALB-SG

VPC ID
[vpc-0286976cf6cb340fe](#)

Owner
600402507299

Inbound rules count
1 Permission entry

Outbound rules count
1 Permission entry

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (1)



Manage tags

Edit inbound rules

Search

< 1 > ⚙

☐	Name	Security group rule...	IP version	Type	Protocol	Port range	Source	Description
☐	-	sg-05d3effebb1efd0f2	-	Custom TCP	TCP	5000	sg-0b305cd088fc2feea / nginx-web-app-sg	-

sg-03bfb6a7b25dcca99 - mysql-sg

Actions

Details

Security group name
mysql-sg

Security group ID
sg-03bfb6a7b25dcca99

Description
mysql-sg

VPC ID
[vpc-0286976cf6cb340fe](#)

Owner
600402507299

Inbound rules count
1 Permission entry

Outbound rules count
1 Permission entry

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (1)



Manage tags

Edit inbound rules

Search

< 1 > ⚙

☐	Name	Security group rule...	IP version	Type	Protocol	Port range	Source	Description
☐	-	sg-018eeeabbd05dc00	-	MYSQL/Aurora	TCP	3306	sg-0f1c65951e8b2df4d / python-backend-api-sg	-

sg-0f1c65951e8b2df4d - python-backend-api-sg

Actions

Details

Security group name
python-backend-api-sg

Security group ID
sg-0f1c65951e8b2df4d

Description
python-backend-api-sg

VPC ID
vpc-0286976cf6cb340fe

Owner
600402507299

Inbound rules count
2 Permission entries

Outbound rules count
1 Permission entry

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (2)



Manage tags

Edit inbound rules

Search

< 1 > ⚙

☐	Name	Security group rule...	IP version	Type	Protocol	Port range	Source	Description
☐	-	sg-04e61e54d5c08b1...	-	Custom TCP	TCP	5000	sg-004ddcb918f8a6f71 / python-app-ALB-SG	-
☐	-	sg-038c38629d3d77...	-	SSH	TCP	22	sg-03885c30b8d03cd55 / bastion-sg	-

sg-03885c30b8d03cd55 - bastion-sg

Actions

Details

Security group name
bastion-sg

Security group ID
sg-03885c30b8d03cd55

Description
bastion-jump-server

VPC ID
vpc-0286976cf6cb340fe

Owner
600402507299

Inbound rules count
1 Permission entry

Outbound rules count
1 Permission entry

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (1)



Manage tags

Edit inbound rules

Search

< 1 > ⚙

☐	Name	Security group rule...	IP version	Type	Protocol	Port range	Source	Description
☐	-	sg-0379bf5e077bf63a8	IPv4	SSH	TCP	22	122.162.149.166/32	-

sg-0b305cd088fc2feea - nginx-web-app-sg

Actions

Details

Security group name
nginx-web-app-sg

Security group ID
sg-0b305cd088fc2feea

Description
nginx-web-app-sg

VPC ID
vpc-0286976cf6cb340fe

Owner
600402507299

Inbound rules count
2 Permission entries

Outbound rules count
1 Permission entry

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (2)



Manage tags

Edit inbound rules

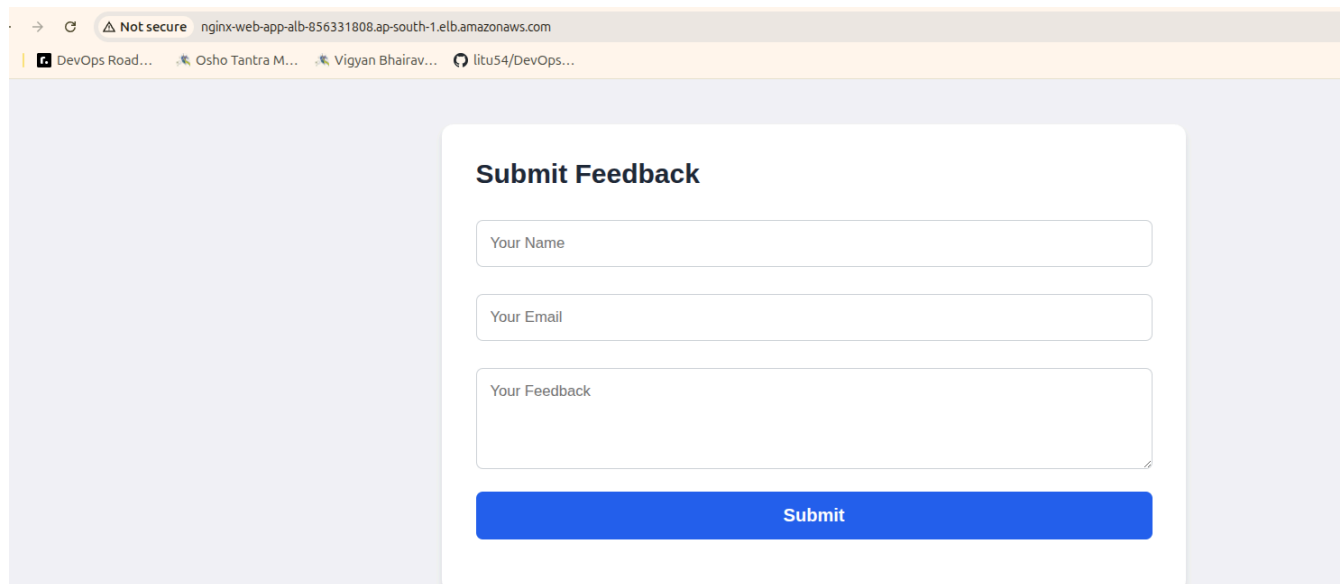
Search

< 1 > ⚙

☐	Name	Security group rule...	IP version	Type	Protocol	Port range	Source	Description
☐	-	sg-0e94d402f334a93c9	-	HTTP	TCP	80	sg-02e908a40ceb9c027 / web-app-ALB	-
☐	-	sg-0415392cc33c55c3f	IPv4	SSH	TCP	22	122.162.149.166/32	-

Task 6: Configure the Internet-Facing Application Load Balancer

This section covers the setup of an Application Load Balancer (ALB) to efficiently distribute incoming public traffic. **A Target Group was created**, and the frontend Web EC2 instances were successfully registered to it. The ALB was securely attached to the Public Subnets, and the application's accessibility was validated by successfully **reaching the frontend application via HTTP using the ALB's default DNS name**



The screenshot shows a web browser window with the address bar displaying 'Not secure' and the URL 'nginx-web-app-alb-856331808.ap-south-1.elb.amazonaws.com'. The browser's tab bar shows several open tabs: 'DevOps Road...', 'Osho Tantra M...', 'Vigyan Bhairav...', and 'litu54/DevOps...'. The main content area features a 'Submit Feedback' form with three input fields: 'Your Name', 'Your Email', and 'Your Feedback'. A blue 'Submit' button is located at the bottom of the form.

nginx-web-app

Actions

Details

arn:aws:elasticloadbalancing:ap-south-1:600402507299:targetgroup/nginx-web-app/ea6f1dde6f6a8b6

Target type

Instance

Protocol : Port

HTTP: 80

Protocol version

HTTP1

VPC

vpc-0286976cf6cb340fe

IP address type

IPv4

Load balancer

nginx-web-app-ALB

1

Total targets

1

Healthy

0

Anomalous

0

Unhealthy

0

Unused

0

Initial

0

Draining

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets

Monitoring

Health checks

Attributes

Tags

Registered targets (1)

Anomaly mitigation: Not applicable

Deregister

Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

< 1 >

Instance ID

Name

Port

Zone

Health status

Health status details

Admini...

Overrid...

Launch ...

Anomaly detectio...

i-Oa7a97f0e242a17b7

nginx-web-app

80

ap-south-1a (a...

Healthy

-

No override...

No overrid...

Septembe...

Normal

nginx-web-app-ALB

Actions

Details

Load balancer type

Application

Status

Active

VPC

vpc-0286976cf6cb340fe

Load balancer IP address type

IPv4

Scheme

Internet-facing

Hosted zone

ZP97RAFLXTNZK

Availability Zones

subnet-000b1eabc355dee4b ap-south-1b (aps1-az3)

Date created

September 16, 2026, 18:10 (UTC+05:30)

subnet-0b7aa218ce03ac985 ap-south-1a (aps1-az1)

Load balancer ARN

arn:aws:elasticloadbalancing:ap-south-1:600402507299:loadbalancer/app/nginx-web-app-ALB/69734893431fdc70

DNS name

nginx-web-app-ALB-856331808.ap-south-1.elb.amazonaws.com (A Record)

Listeners and rules

Network mapping

Resource map

Security

Monitoring

Integrations

Attributes

Capacity

Tags

Listeners and rules (1)

Manage rules

Manage listener

Add listener

A listener checks for connection requests on its configured protocol and port. Incoming requests are evaluated against rules in priority order; the first matching rule determines routing. If no rule matches, the default action is applied.

Filter listeners

< 1 >

Protocol:Port

Default action

Rules

ARN

Security policy

Default SSL/TLS certificate

mTLS

Trust store

HTTP:80

Forward to target group

nginx-web-app: 1 (100%)

Target group stickiness: Off

1 rule

ARN

Not applicable

Not applicable

Not applicable

Not applica

Task 7: Configure Internal Application Load Balancer

This section describes the deployment of an Internal Application Load Balancer (ALB) to securely manage and route traffic between the frontend and backend layers. An internal ALB was provisioned along with a dedicated Target Group configured with appropriate health checks. The backend Flask EC2 instances (Application Tier) were successfully registered to this Target Group. Finally, seamless and secure internal communication was validated to ensure that the Web Tier can successfully route requests to the Application Tier.

```
ubuntu@ip-10-0-1-4:~$ curl -X GET internal-python-backend-ALB-internal-773946408.ap-south-1.elb.amazonaws.com:5000
{"status":"Flask is running"}
ubuntu@ip-10-0-1-4:~$
```

python-backend-tg

Actions

Details

arn:aws:elasticloadbalancing:ap-south-1:600402507299:targetgroup/python-backend-tg/ca9e0feb892bb93

Target type

Instance

Protocol : Port

HTTP: 5000

Protocol version

HTTP1

VPC

vpc-0286976cf6cb340fe

IP address type

IPv4

Load balancer

python-backend-ALB-internal

1

Total targets

1

Healthy

0

Anomalous

0

Unhealthy

0

Unused

0

Initial

0

Draining

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets

Monitoring

Health checks

Attributes

Tags

Registered targets (1)

Anomaly mitigation: Not applicable

Deregister

Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

< 1 >

☐	Instance ID	Name	Port	Zone	Health status	Health status details	Administrative o...	Override details	Launch...	Anon
☐	i-0b6044a41de38fc0c	python-backend-api	5000	ap-south-1b (a...	Healthy	-	No override	No override is curre...	Septembe...	Ne

python-backend-ALB-internal

Actions

Details

Load balancer type

Application

Status

Active

VPC

vpc-0286976cf6cb340fe

Load balancer IP address type

IPv4

Scheme

Internal

Hosted zone

ZP97RAFLXTNZK

Availability Zones

subnet-032cab7b44490b9f ap-south-1b (aps1-az3)
subnet-04b444f224fdb8e2d ap-south-1a (aps1-az1)

Date created

September 16, 2026, 19:00 (UTC+05:30)

Load balancer ARN

arn:aws:elasticloadbalancing:ap-south-1:600402507299:loadbalancer/app/python-backend-ALB-internal/ebb69b1a121cbd73

DNS name

internal-python-backend-ALB-internal-773946408.ap-south-1.elb.amazonaws.com (A Record)

Listeners and rules

Network mapping

Resource map

Security

Monitoring

Integrations

Attributes

Capacity

Tags

Listeners and rules (1)

Manage rules

Manage listener

Add listener

A listener checks for connection requests on its configured protocol and port. Incoming requests are evaluated against rules in priority order; the first matching rule determines routing. If no rule matches, the default action is applied.

Filter listeners

< 1 >

☐	Protocol:Port	Default action	Rules	ARN	Security policy	Default SSL/TLS certificate	mTLS	Trust store
☐	HTTP:5000	Forward to target group python-backend-tg: 1 (100%) Target group stickiness: Off	1 rule	ARN	Not applicable	Not applicable	Not applicable	Not applic

Task 8: Configure Nginx Reverse Proxy for API Routing

This section covers the configuration of the Nginx web server to act as a reverse proxy.

The default Nginx configuration was replaced with the custom `nginx.conf` provided in the project repository. This configuration was updated to securely route all backend API requests from the frontend Web Tier directly to the Internal Application Load Balancer. The Nginx service was subsequently restarted, and successful end-to-end communication between the frontend and backend tiers was verified.

```
ubuntu@ip-10-0-1-4:~$ sudo systemctl restart nginx.service
ubuntu@ip-10-0-1-4:~$ _
```

```
ubuntu@ip-10-0-1-4:~$ cat /etc/nginx/nginx.conf
user www-data;
worker_processes auto;
pid /run/nginx.pid;
error_log /var/log/nginx/error.log;
include /etc/nginx/modules-enabled/*.conf;

events {
    worker_connections 768;
    # multi_accept on;
}

http {

    ##
    # Basic Settings
    ##

    sendfile on;
    tcp_nopush on;
    types_hash_max_size 2048;
    # server_tokens off;

    # server_names_hash_bucket_size 64;
    # server_name_in_redirect off;

    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    server {

        listen 80;

        server_name _;

        root /usr/share/nginx/html;

        index index.html;

        location / {
            try_files $uri $uri/ =404;
        }

        location /api/ {
            proxy_pass http://internal-python-backend-ALB-internal-773946408.ap-south-1.elb.amazonaws.com:5000/;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }
    }
}
```

Task 9 & 10: Integration and Validation

This section focuses on the final integration between the backend Application Tier and the Database Tier. The Flask application's configuration was updated with the accurate Amazon RDS connection details. Following this update, the successful connection between the application and the database was verified. To validate the complete end-to-end data flow, sample feedback was submitted through the frontend web application, and it was confirmed that the submitted data was successfully processed and stored within the database.

Name	X	Headers	Payload	Preview	Response	Initiator	Timing
submit	1				{ "message": "Feedback submitted successfully!" }		
feedback	2						

Name	X	Headers	Payload	Preview	Response	Initiator	Timing
submit							
feedback							

Request Payload

View source

```
{name: "Kapil Singh", email: "hhhhh@gmail.com", feedback: "ggjjgj"}
email: "hhhhh@gmail.com"
feedback: "ggjjgj"
name: "Kapil Singh"
```

Name	X	Headers	Payload	Preview	Response	Initiator	Timing
(i) submit							
(i) feedback							
▼ General							
Request URL		http://nginx-web-app-alb-856331808.ap-south-1.elb.amazonaws.com/api/submit					
Request Method		POST					
Status Code		200 OK					
Remote Address		3.109.146.161:80					
Referrer Policy		strict-origin-when-cross-origin					
▼ Response headers ☐ Raw							
Connection		keep-alive					
Content-Length		47					
Content-Type		application/json					
Date		Wed, 16 Sep 2026 19:44:10 GMT					
Server		nginx/1.24.0 (Ubuntu)					

```
ubuntu@ip-10-0-44-194:~$ ls
app.py  venv
ubuntu@ip-10-0-44-194:~$ cat app.py
from flask import Flask, request, jsonify
import pymysql

app = Flask(__name__)

DB_CONFIG = {
    'host': 'guvi-project1-mysql-db.c7q0e0ku8lf1.ap-south-1.rds.amazonaws.com',
    'user': 'admin',
    'password': 'Nevergiveup1000',
    'db': 'feedbackdb',
    'cursorclass': pymysql.cursors.DictCursor
}

@app.route('/')
def health():
    return jsonify({'status': 'Flask is running'})
```

Additional Task: Auto Scaling

This section outlines the setup of an Auto Scaling Group (ASG) to ensure high availability for the Nginx web application. A customized Launch Template was created, defining the AML, instance type, and Security Groups.

nginx-web-app-launch-template (lt-0ef679c8e265b7f76)

ActionsDelete template

Launch template details

Launch template ID
lt-0ef679c8e265b7f76

Launch template name
nginx-web-app-launch-template

Default version
1

Owner
arn:aws:iam::600402507299:root

DetailsVersionsTemplate tags

Launch template version details

ActionsDelete template version

Version
1 (Default)

Description
nginx-web-app-launch-template

Date created
2026-09-16T20:00:55.000Z

Created by
arn:aws:iam::600402507299:root

Instance detailsStorageResource tagsNetwork interfacesAdvanced details

AMI ID
ami-006f82a1d5a27da54

Instance type
t3.micro

Availability Zone
-

Availability Zone Id
-

Key pair name
mumbai-key-sep-16

Security groups
-

Security group IDs
sg-0b305cd088fc2feeaa

nginx-web-app-autoscaling

nginx-web-app-autoscaling Capacity overviewEdit

arn:aws:autoscaling:ap-south-1:600402507299:autoScalingGroup:8fbd1e31-fdb2-4913-8e93-681061098bb0:autoScalingGroupName/nginx-web-app-autoscaling

Desired capacity
1

Scaling limits
1 - 3

Desired capacity type
Units (number of instances)

Status
At desired capacity

Date created
Thu Sep 17 2026 01:33:40 GMT+0530 (India Standard Time)

DetailsIntegrationsAutomatic scalingInstance managementInstance refreshActivityMonitoringTags

Launch template

Edit

Launch template
lt-0ef679c8e265b7f76
nginx-web-app-launch-template

AMI ID
ami-006f82a1d5a27da54

Instance type
t3.micro

Owner
arn:aws:iam::600402507299:root

Version
Default

Security groups
-

Security group IDs
sg-0b305cd088fc2feeaa

Create time
Thu Sep 17 2026 01:30:55 GMT+0530 (India Standard Time)

Description
nginx-web-app-launch-template

Storage (volumes)
-

Key pair name
mumbai-key-sep-16

Request Spot Instances
No

View details in the launch template console

Network

Edit

Availability Zones
aps1-a23 (ap-south-1b)
aps1-a21 (ap-south-1a)

Subnet ID
subnet-000b1eabc355dee4b

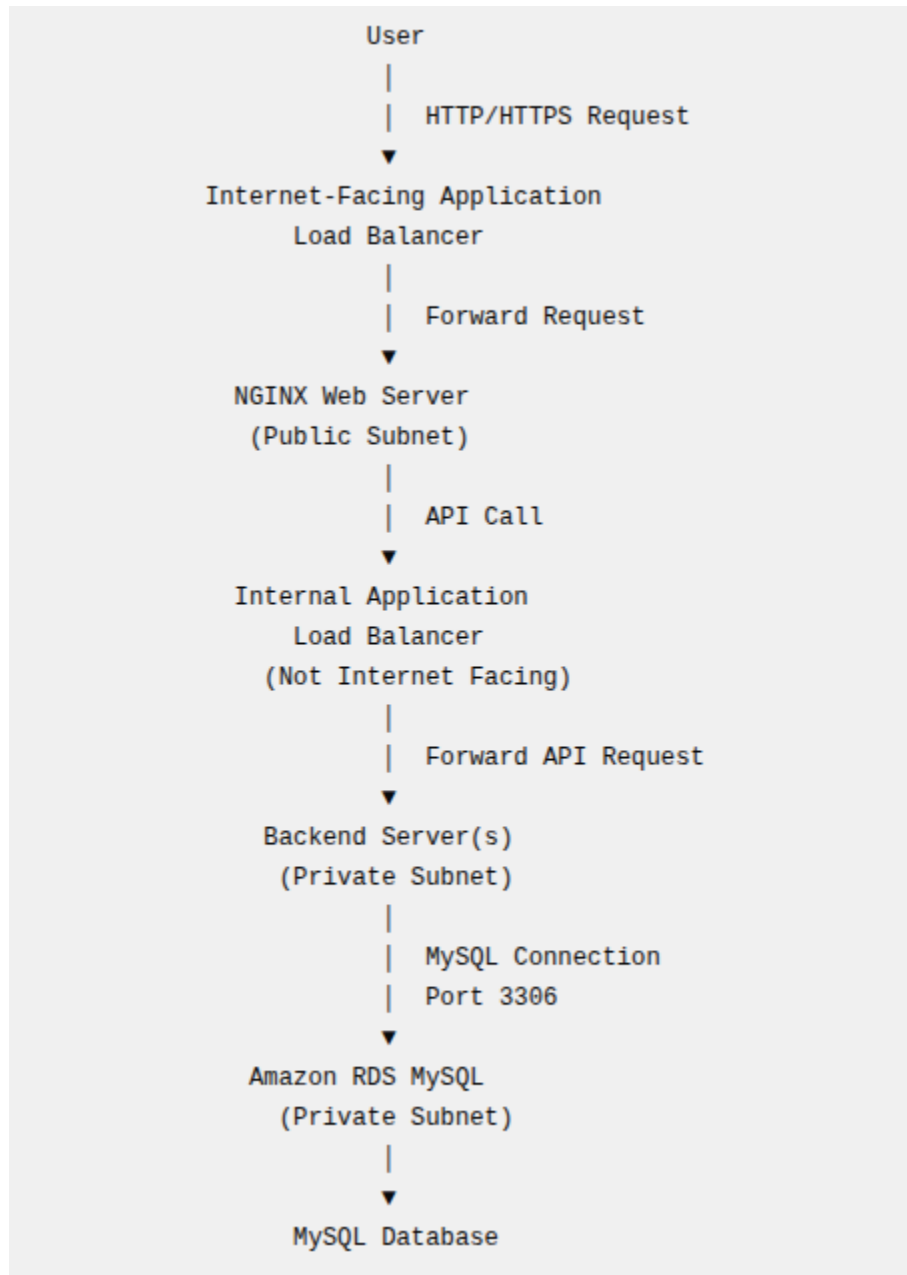
Availability Zone distribution
Balanced best effort

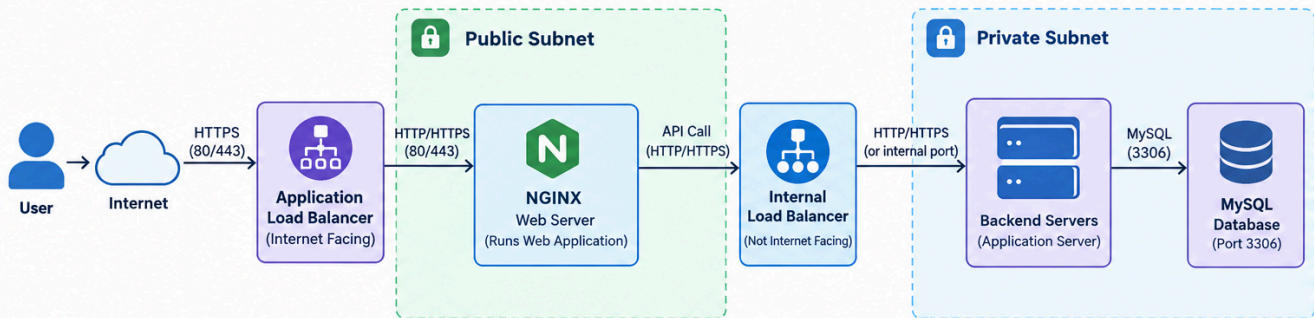
SQL queries to get stored Data

USE feedbackdb;

SELECT * FROM feedback;

Architecture Flow Diagram





Flow

